

Politechnika Śląska

Wydział Automatyki, Elektroniki i Informatyki

Wprowadzenie do projektowania mikroprocesorów

Mikroinformatyka Systemów Cyfrowych S2, semestr 2

Autor:

Łukasz Wolny

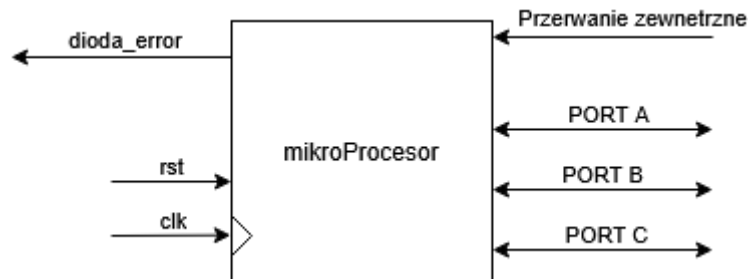
Krótką dokumentacją + programy + wymagania.

Spis treści

1-OPIS	2
2-Programy	5
3-REQ	14

1-OPIS

1.1. Mikroprocesor



8-bitowy mikroprocesor.

3 porty wej/wyj.

Dioda error do sygnalizowania wyjątku(przepiętnienie stosu)

Przerwanie zewnętrzne do symulowania zewnętrznego przerwania (przycisk).

1.2. Rozkazy

		x13,x12,x11							
		000	001	011	010	110	111	101	100
x15,x14	00	RST	LD	ST	AND	XOR	JZ	OR	JMP
	01	? NOP	ADD	JNZ	SUB	NOT	JP	INC	JC
	11	SEI	CLI	TIMER_SET_L	RETI	TIMER_SET_H	NOP	TIMER_CTRL	TIMER_READ
	10	ADDC	JS	SUBC	JOV	CALL	RET	POP	PUSH

Jeden adres został niewykorzystany – 01000 – siłą rzeczy stał się NOP.

Rozkazy 16bitowe.

15	11	10	8	7	0
Opcode			Operand-type		Operand

Operand-type		Operand
LD	000	address
	001	rejestr
	010	PIN
	011	IM(zmienna)
	100	@Rx
ST	000	address
	001	rejestr
	010	PORT
	011	DDR

Rozkazy posiadają maksymalnie jeden argument (np. LD R7).

RST: (bez argumentu) reset procesora (resetuje licznik rozkazów).

LD: (z argumentem) załaduj do akumulatora odpowiednią wartość (operand-type).

ST: (z argumentem) zapisz wartość z akumulatora do odpowiedniego miejsca (operand-type).

NOP: (bez argumentu) „nic nie rób”. Licznik rozkazów się nie inkrementuje.

PUSH/POP: (bez argumentu) zapisz/zdejmij ze stosu wartość z/do akumulatora.

SEI/CLI: (bez argumentu) włącza/wyłącza przerwania.

RETI: powrót z obsługi przerwania.

1.2.1. ALU

Operacje arytmetyczne/logiczne na ALU wykonywane są tylko między akumulatorem a rejestrami. NOT oraz INC dotyczy tylko Acc. Operacja LD przepisuje z wejścia(Rx, MEM,...) do Acc.

AND Rx, XOR Rx, OR Rx, ADD Rx, SUB Rx, NOT, INC, ADDC Rx, SUBC Rx.

Np. ADD R2. $Acc \leftarrow R2 + Acc$

NOT. $Acc \leftarrow \sim Acc$

1.2.2. Skok

Operacje skoków, wykonują skok do wskazanego adresu w pamięci programu (zapis adresu do pc).

JMP, JZ, JNZ, JP, JC, TIMER_READ, JS, JOV.

JMP – podstawowy, wykonuje skok pod wskazany adres. Reszta wykonuje skok w zależności od ustawienia konkretnej flagi – jeśli jest ustawiona odpowiednia flaga to wykonaj skok, jeśli nie wykonaj kolejną instrukcję.

CALL: skok do podprogramu. Wartość licznika rozkazów zapisuje się na stosie.

RET: powrót z podprogramu. Wartość ze stosu wraca do licznika rozkazów.

1.2.3. Licznik

TIMER_SET_L/TIMER_SET_H: zapis wartości 8-bitowych do licznika(16bitowego), rozdzielona na dwie części.

TIMER_CTRL: zapis do licznika rejestru sterującego.

7	6	5	4	3	2	1	0
enable	X	X	Tryb	int_enable	Preskaler_3	Preskaler_2	Preskaler_1

1.3. Program

Każdy program musi zacząć się w ten sposób.

```
0x00 JMP start
0x02 przerwanie ext
0x04 przerwanie licznik
0x06 (wyjątek) JMP 0x06
0x08 start:
```

...

main:

Dla podprogramów (przerwania i CALL) należy zapisać wartość Acc (mikroprocesor tego nie zapewnia). Np.:

0x200: podprogram

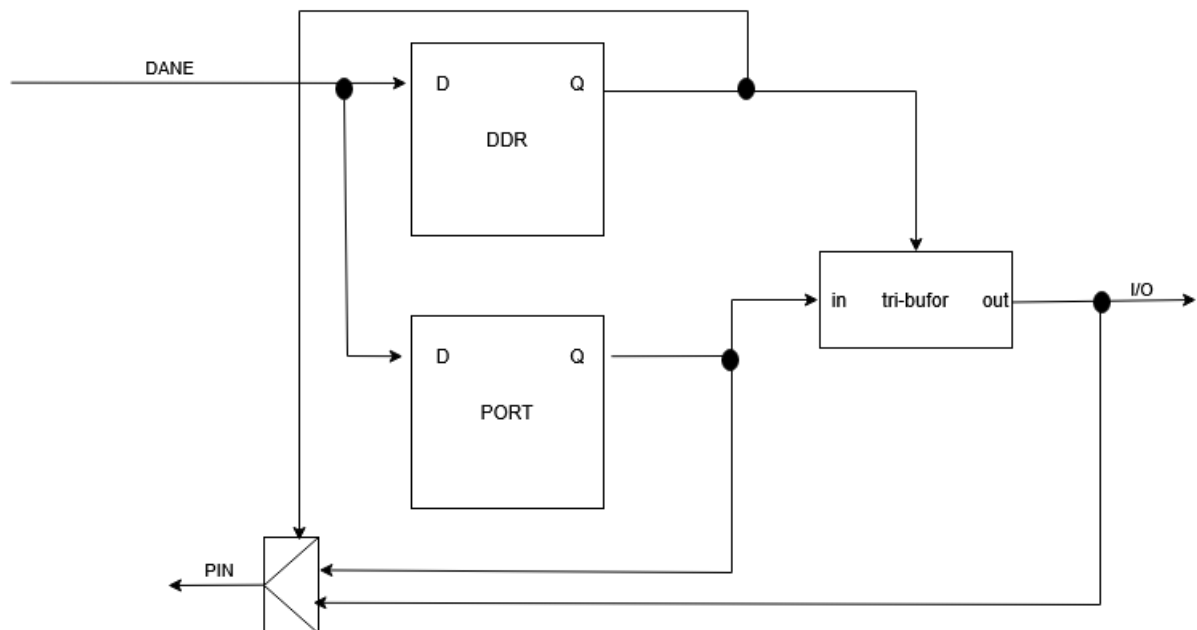
PUSH

.....

POP

RET/RETI

1.4. Porty



Rejestr kierunku DDRx: 1 = wyjście. 0 = wejście.

1.5. Przerwanie + licznik

Licznik jest 16bitowy. Dostępne są 2 tryby. Pierwszy gdzie licznik liczy do maksymalnej wartości (16bitowej 0xFFFF), oraz drugi gdzie licznik umożliwia ustawienie maksymalnej wartości. Dostępne są preskalery: 8,64,256,1024.

Licznik zawsze generuje flagę, oraz możliwość generowania przerwania.

Dostępny jest układ przerwań z wejściami dla zewnętrznego przerwania oraz przerwania z licznika. Przerwanie reaguje na narastające zbocze sygnału. Priorytet ma zawsze przerwanie zewnętrzne.

1.6. Stos

W przypadku przepełnienia/niedomiaru stosu (wyjątek) procesor zatrzymuje wykonywanie dalszych rozkazów i sygnalizuje to diodą error.

1.7. Flagi

Do rejestru flag nowe wartości zapisywane są tylko w momencie kiedy następuje zapis do akumulatora.

Jeśli wystąpiła operacja arytmetyczna to aktualizowane są odpowiednie flagi (C, OV). Jeśli nastąpiła operacja logiczna, to te flagi są kasowane.

1.8. Pamięć danych

Pamięć dla danych ma możliwość wyboru stron.

Rejestr stron znajduje się pod adresem 255.

Dla 16 stron, dostępna pamięć do 4KB.

2-Programy

tb/procesor_tb.sv

1. Program_1

Test tego przycisku jednego w porcie i tutaj widać drgania...

```
JMP main
Main: LD #0;
ST DDR0;
```

```

LD #8'b11111110
ST DDR1;
LD #8'b11111111;
ST DDR2
LD #8'b11111110
ST R7
LD #0
ST R0
ST R1
ST R2
(20) main:
LD PIN1
OR R7
NOT
JZ przycisk
LD R0
ST R2
(26)dalej:
LD R1
ST PORT2
JMP main
RST
(30) przycisk:
LD R2
JNZ jmp: back
INC
ST R2
LD R1
INC
ST R1
(37)back: JMP dalej
RST

```

2. Program_2

to samo ale z licznikiem 20ms do usunięcia drgań

licznik + licznik_flaga

```

JMP start
(8)start:LD #0
ST DDR0;
LD # 8'b11111110 -> 0 - przycisk
ST DDR1; //B - nic + 1 przycisk
LD #8'b11111111;
ST DDR2; //C - diody
LD # 8'b11111110
ST R7
LD # 0 8'b00000000

```

```

ST R0
ST R1
ST R2
(20) main: LD PIN1
OR R7
NOT
IF Z (JZ) skok do: przycisk
LD R0
ST R2
//dalsza czesc programu
(26)dalej:
...
LD R1
ST PORT2
...
JMP main
RST

(30) przycisk:
//tutaj ten licznik 20ms
TIMER_CTRL 00000011 (off)
TIMER_SET_H 0111 1010
TIMER_SET_L 0001 0010
TIMER_CTRL 10000011 (on)

//czekam na flage
(34) TIMER_READ jest_flaga
JMP 34 0010 0010
(36)jest_flaga: LD PIN1
OR R7
NOT

IF Z (JNZ) skok do: back
(40)LD R2
JNZ jmp: back
INC
ST R2
LD R1
INC
ST R1
(47)back: JMP dalej
RST

```

3. Program_3

To samo tylko ze na 7seg
 cyfry od 0 do 15 zapisane kod 7seg w pamieci na 1 stronie.
 adres do MEM po pośrednim (przez Rx)

```

//---
JMP main
(8):
LD #0;8'b000000000
ST DDR0; //A - SW
LD # 8'b111111110 -> 0 - przycisk
ST DDR1; //B - nic + 1 przycisk
LD #8'b111111111; //LEed
ST DDR2; //C - diody
LD # 8'b111111110
ST R7 - do R7 potrzebne do operacji OR
LD # 0 8'b00000000
ST R0
ST R1 //to bedzie spr czy przycisk jest wcisniety
ST R2
//7seg kody
LD #1 00000001
ST R6 //rejestr do nr stron przechowuje
ST 255 - pod adres 255(zmiana strony na 1)
LD # 00000001 //0
ST 0
LD # 01001111 //1
ST 1
LD # 00010010 //2
ST 2
LD # 00000110 //3
ST 3
LD # 01001100 //4
ST 4
LD # 00100100 //5
ST 5
LD # 00100000 //6
ST 6
LD # 00001111 //7
ST 7
LD # 00000000 //8
ST 8
LD # 00000100 //9
ST 9
LD # 00001000 //A
ST 10
LD # 01100000 //B
ST 11
LD # 00110001 //C
ST 12

```



```

LD # 01000010 //D
ST 13
LD # 00110000 //E
ST 14
LD # 00111000 //F
ST 15
LD R0
ST R6
ST 255 -
(58) main:
OR R7
NOT
IF Z (JZ) skok do: przycisk
LD R0
ST R2 //reset R2.
//dalsza czesc programu
(64)dalej:
...
LD #00001111
AND R1 -maska AND R1 = 4bity 0-15
ST R1
LD #00000001
ST 255

LD #00000000
ST PORT2

LD @R1
ST PORT2
...
JMP main
RST
(75) przycisk:
//tutaj ten licznik 20ms
TIMER_CTRL 00000011
TIMER_SET_H 0111 1010
TIMER_SET_L 0001 0010
TIMER_CTRL 10000011

(79) TIMER_READ jest_flaga
JMP 79 0100 1101
(81)jest_flaga: LD PIN1
OR R7
NOT

IF Z (JNZ) skok do: back
(85)LD R2
JNZ jmp: back

```

```

INC
ST R2
LD R1
INC
ST R1
(92)back: JMP dalej
RST

```

4. Program_4

push/pop -> error dioda (bez 7seg)

```

JMP start
00000 000 000000000
00000 000 000000000
00000 000 000000000
00000 000 000000000
00000 000 000000000
JMP 0x06
00000 000 000000000
(start): LD #0;
ST DDR0; //A - SW
LD #8'b11111110 -> 0 - przycisk
ST DDR1; //B - nic + 1 przycisk
LD #8'b11111111; //LEed
ST DDR2; //C - diody
LD #8'b11111110
ST R7
LD #0 8'b00000000
ST R0
ST R1
ST R2
(20) main:
LD PIN1
OR R7
NOT
IF Z (JZ) skok do: przycisk
LD R0
ST R2 //reset R2.
(26)dalej:
...
LD R1
ST PORT2
...
JMP main
RST

```

(30) przycisk:

```

//tutaj ten licznik 20ms
TIMER_CTRL off
TIMER_SET_H 0111 1010
TIMER_SET_L 0001 0010
TIMER_CTRL on

(34) IF(flaga_licznik) TIMER_READ jest_flaga
JMP 34
(36) jest_flaga:
LD PIN1 //sprawdzanie tego przycisku
OR R7
NOT
IF Z (JNZ) skok do: back
(40) LD R2
JNZ jmp: back
INC
ST R2
LD R1
INC
ST R1
PUSH
(48) back: JMP dalej
RST

```

5. Program_5

przerwania zew+licznik
 licznik co 1s(licznik zlicza 500ms. 2x=1s) inkrementuje (7seg). przerwanie zeruje

```

JMP start
00000 000 000000000
JMP przerwanie_ext (100)
00000 000 000000000
JMP przerwanie_licznik (200)
00000 000 000000000
JMP 0x06
00000 000 000000000
Start: LD #0;
ST DDR0; //A - SW
LD #8'b11111110 -> 0 - przycisk
ST DDR1; //B - nic + 1 przycisk
LD #8'b11111111; //LEed
ST DDR2; //C - diody
LD #8'b00000010
ST R7 - do R7 potrzebne do 1s sprawdzania. zaladowane 2.
LD #0 8'b00000000
ST R0

```

```

ST R1 //to bedzie spr czy przycisk jest wcisniety
ST R2
//7seg kody
LD #1 00000001
ST R6 //rejestr do nr stron przechowuje
ST 255 - pod adres 255(zmiana strony na 1)
LD # 00000001 //0
ST 0
LD # 01001111 //1
ST 1
LD # 00010010 //2
ST 2
LD # 00000110 //3
ST 3
LD # 01001100 //4
ST 4
LD # 00100100 //5
ST 5
LD # 00100000 //6
ST 6
LD # 00001111 //7
ST 7
LD # 00000000 //8
ST 8
LD # 00000100 //9
ST 9
LD # 00001000 //A
ST 10
LD # 01100000 //B
ST 11
LD # 00110001 //C
ST 12
LD # 01000010 //D
ST 13
LD # 00110000 //E
ST 14
LD # 00111000 //F
ST 15
LD R0
ST R6
ST 255 - zmiana strony spowrotem na 0.
LD R2
INC
ST R2 //R2 = 1
----
sei
TIMER_CTRL off
TIMER_SET_H 1011 1110

```

TIMER_SET_L 1011 1100
TIMER_CTRL on + ext int en

(66) main:

LD R7
JNZ R7 skok adres +2 (69)
CALL inkrementacja_podprogram
LD #00001111 --zaladowanie maski
AND R1 -maska AND R1 = 4bity 0-15
ST R1 - zapisz -> w R1 jest adres seg do wys
LD #00000001
ST 255 - zmiana na 1 strone
LD @R1 - zaladuj znak
ST PORT2 -> na 7seg
LD #00000000
ST 255 - zmiana na 0 strone
JMP main
RST

---- gdzies tam w pamieci --:

(100)ext przerwanie:

PUSH
LD #00000000
ST R1
POP
RETI
RST

(200)ext licznik: R7 - 1

PUSH
LD R7
SUB R2
ST R7
POP
RETI
RST

(150) inkrementacja_podprogram:

PUSH
LD R1
INC
ST R1
LD R7
inc
inc
ST R7
POP
RET
RST

3-REQ

1. ALU.

REQ_ALU:

REQ_ALU_1:

Moduł musi realizować operacje arytmetyczno-logiczne na danych wejściowych a i b o szerokości określonej parametrem ALU_rozm_data.

REQ_ALU_2:

Wybór operacji wykonywanej przez ALU musi być realizowany na podstawie 4-bitowego sygnału sterującego alu_op.

REQ_ALU_3:

Dla kodu operacji alu_op = 0000 moduł musi przekazywać dane z wejścia b bez modyfikacji na wyjście out.

REQ_ALU_4:

Dla kodów operacji alu_op = 0001, 0010 oraz 0011 moduł musi realizować odpowiednio operacje logiczne AND, OR oraz XOR na wejściach a i b.

REQ_ALU_5:

Dla kodu operacji alu_op = 0100 moduł musi realizować operację dodawania ($a + b$) z generacją flagi przeniesienia.

REQ_ALU_6:

Dla kodu operacji alu_op = 0101 moduł musi realizować operację odejmowania ($a - b$) z generacją flagi przeniesienia.

REQ_ALU_7:

Dla kodu operacji alu_op = 0110 moduł musi realizować inkrementację wartości wejścia a.

REQ_ALU_8:

Dla kodu operacji alu_op = 0111 moduł musi realizować operację negacji bitowej wejścia a.

REQ_ALU_9:

Dla kodów operacji alu_op = 1000 oraz 1001 moduł musi realizować odpowiednio operacje dodawania i odejmowania z przeniesieniem, wykorzystując sygnał wejściowy C_in.

REQ_ALU_10:

Moduł musi generować flagę zera (Z), która jest ustawiona, gdy wynik operacji jest równy zero.

REQ_ALU_11:

Moduł musi generować flagę znaku (S) na podstawie najstarszego bitu wyniku operacji.

REQ_ALU_12:

Moduł musi generować flagę parzystości (P), która jest ustawiona, gdy liczba jedynek w wyniku operacji jest parzysta.

REQ_ALU_13:

Moduł musi generować flagę przeniesienia (C) dla operacji arytmetycznych.

REQ_ALU_14:

Moduł musi generować flagę przepiętnienia arytmetycznego (OV) zgodnie z zasadami arytmetyki ze znakiem.

2. Akumulator.

REQ_ACC:

REQ_ACC_1:

Moduł musi realizować rejestr danych o szerokości określonej parametrem ALU_rozm_data.

REQ_ACC_2:

Po aktywacji sygnału reset (rst) zawartość akumulatora musi zostać wyzerowana w jednym cyklu zegarowym.

REQ_ACC_3:

Jeżeli sygnał zapisu akumulatora (A_ce) jest aktywny, moduł musi zapisać wartość z wejścia a do akumulatora przy narastającym zboczach sygnału clk.

REQ_ACC_4:

Jeżeli sygnał A_ce jest nieaktywny, zawartość akumulatora musi pozostać niezmienną.

REQ_ACC_5:

Aktualna zawartość akumulatora musi być stale dostępna na wyjściu out.

REQ_ACC_6:

Wszystkie operacje zapisu oraz resetowania akumulatora muszą być realizowane synchronicznie z narastającym zboczem sygnału zegarowego clk.

3. Dekoder rozkazów.

REQ_Dekoder:

REQ_Dekoder_1:

Moduł ID musi dekodować instrukcję maszynową na podstawie pola operacji zawartego w rozkazie.

REQ_Dekoder_2:

Moduł ID musi generować sygnały sterujące umożliwiające wykonanie operacji arytmetycznych i logicznych przez jednostkę ALU.

REQ_Dekoder_3:

Moduł ID musi umożliwiać wybór źródła danych operandu, w tym wartości natychmiastowej, rejestru, pamięci oraz portów wejścia–wyjścia.

REQ_Dekoder_4:

Moduł ID musi obsługiwać instrukcje zapisu i odczytu danych do rejestrów, pamięci oraz portów I/O.

REQ_Dekoder_5:

Moduł ID musi obsługiwać instrukcje skoku bezwarunkowego oraz warunkowego w oparciu o aktualny stan flag procesora.

REQ_Dekoder_6:

Moduł ID musi obsługiwać mechanizm stosu danych oraz stosu adresów powrotu, w tym wykrywanie przepełnienia i niedomiaru stosu.

REQ_Dekoder_7:

Moduł ID musi obsługiwać system przerwań, w tym przyjmowanie przerwania, zapis adresu powrotu na stos oraz skok do wektora przerwania.

REQ_Dekoder_8:

Moduł ID musi umożliwiać konfigurację i obsługę licznika czasowego, w tym zapis rejestrów sterujących oraz reakcję na flagę licznika.

REQ_Dekoder_9:

Moduł ID musi generować sygnały sterujące flagami procesora w zależności od typu wykonywanej instrukcji.

REQ_Dekoder_10:

W przypadku wykrycia sytuacji wyjątkowych, takich jak przepełnienie stosu, moduł ID musi wygenerować odpowiednią reakcję systemową.

4. Rejestry.

REQ_Rx:

REQ_Rx_1:

Moduł musi posiadać Rx_liczba rejestrów, każdy o szerokości Rx_rozm_data bitów.

REQ_Rx_2:

Podczas resetu (rst = 1) wszystkie rejestry muszą zostać wyzerowane (wartość 0).

REQ_Rx_3:

Jeżeli wr_Rx = 1, to podczas narastającego zbocza zegara clk wartość wejściowa dane musi zostać zapisana do rejestru o numerze nr_Rx.

REQ_Rx_4:

Jeżeli wr_Rx = 0, to żaden rejestr nie może zostać zmieniony.

REQ_Rx_5:

Odczyt rejestru musi być wykonywany asynchronicznie, a wyjście out musi zawsze odzwierciedlać aktualną zawartość rejestru rejestr[nr_Rx].

5. Flagi.

REQ_Flagi:

REQ_Flagi_1:

Moduł musi przechowywać zestaw flag statusowych procesora w postaci rejestru synchronizowanego sygnałem zegarowym clk.

REQ_Flagi_2:

Po aktywacji sygnału reset (rst) wszystkie flagi statusowe muszą zostać wyzerowane w jednym cyklu zegarowym.

REQ_Flagi_3:

Aktualizacja flag statusowych musi następować wyłącznie wtedy, gdy sygnał zapisu flag (flagi_en) jest aktywny.

REQ_Flagi_4:

Dla operacji arytmetycznych, przy aktywnym sygnale C_OV_en, moduł musi aktualizować flagę przeniesienia (C) oraz flagę przepełnienia (OV) na podstawie sygnałów wejściowych C_in i OV_in.

REQ_Flagi_5:

Dla operacji logicznych, przy aktywnym sygnale C_OV_kasowanie, moduł musi kasować flagę przeniesienia (C) oraz flagę przepełnienia (OV).

REQ_Flagi_6:

Flagi parzystości (P), zera (Z) oraz znaku (S) muszą być aktualizowane na podstawie sygnałów wejściowych P_in, Z_in i S_in przy każdej aktywacji zapisu flag.

REQ_Flagi_7:

Jeżeli sygnał flagi_en jest nieaktywny, stan wszystkich flag musi pozostać niezmienny.

REQ_Flagi_8:

Aktualny stan każdej z flag musi być stale dostępny na odpowiednich wyjściach modułu.

REQ_Flagi_9:

Wszystkie operacje zapisu i kasowania flag muszą być realizowane synchronicznie z narastającym zboczem sygnału zegarowego clk.

6. Licznik

REQ_Licznik:

REQ_Licznik_1:

Moduł musi realizować 16-bitowy licznik taktowany sygnałem clk, którego praca jest sterowana sygnałem licznik_enable.

REQ_Licznik_2:

Po aktywacji sygnału reset (rst) licznik, preskaler, flagi oraz rejestry konfiguracyjne muszą zostać wyzerowane w jednym cyklu zegarowym.

REQ_Licznik_3:

Moduł musi umożliwiać konfigurację preskalera, trybu pracy, włączenia licznika oraz włączenia przerwań poprzez zapis do rejestru sterującego (zapisz_ctr).

REQ_Licznik_4:

Moduł musi umożliwiać zapis 16-bitowej wartości progowej licznika (wartosc_max) w dwóch etapach: dolnego bajtu (zapisz_L) oraz górnego bajtu (zapisz_H).

REQ_Licznik_5:

Jeżeli licznik jest włączony (licznik_enable = 1), moduł musi inkrementować licznik zgodnie z ustawioną wartością preskalera.

REQ_Licznik_6:

W trybie przepiętnienia (tryb = 1) moduł musi generować zdarzenie po osiągnięciu przez licznik wartości maksymalnej (0xFFFF), zerować licznik oraz ustawiać flagę licznik_flaga.

REQ_Licznik_7:

W trybie porównania (tryb = 0) moduł musi generować zdarzenie po osiągnięciu przez licznik zaprogramowanej wartości maksymalnej (wartosc_max), zerować licznik oraz ustawiać flagę licznik_flaga.

REQ_Licznik_8:

W przypadku wystąpienia zdarzenia licznika, moduł musi wygenerować sygnał przerwania (licznik_int) tylko wtedy, gdy przerwania są włączone (int_enable = 1).

REQ_Licznik_9:

Flaga zdarzenia licznika (licznik_flaga) musi pozostać ustawiona do momentu jej skasowania sygnałem licznik_flaga_clear.

REQ_Licznik_10:

Jeżeli licznik jest wyłączony (licznik_enable = 0), licznik oraz sygnały przerwania i flagi muszą pozostawać wyzerowane.

REQ_Licznik_11:

Wszystkie operacje modułu muszą być realizowane synchronicznie z narastającym zboczem sygnału zegarowego clk.

7. Pamięć_data.

REQ_MEM:

REQ_MEM_1:

Fizyczna pamięć ma rozmiar $2^{(ADDR_WIDTH_MEM + DATA_WIDTH_STRONY)}$ słów.

REQ_MEM_2:

Wejście adres określa adres logiczny (offset) w zakresie $[0, 2^{\text{ADDR_WIDTH_MEM}} - 1]$.

REQ_MEM_3:

Moduł zawiera rejestr strony strona o szerokości DATA_WIDTH_STRONY, który jest używany do mapowania logicznego adresu na fizyczny.

REQ_MEM_4:

Rejestr strony jest aktualizowany tylko podczas zapisu (sygnał wr_mem = 1) do specjalnego adresu 255.

REQ_MEM_5:

Dla adresu 255 (specjalny adres rejestru stron) moduł powinien zwracać na wyjściu out aktualną wartość rejestru strony.

REQ_MEM_6:

Jeśli wr_mem = 1 i adres != 255, następuje zapis dane do pamięci pod adresem fizycznym {strona, adres}.

REQ_MEM_7:

Odczyt pamięci jest asynchroniczny i zawsze zwraca aktualną zawartość pod adresem fizycznym {strona, adres}, z wyjątkiem adresu 255 (REQ_MEM_6).

REQ_MEM_8:

Jeśli rst = 1, rejestr strony strona jest zerowany (do wartości 0). Reset nie wpływa na zawartość pamięci mem.

8. Pamięć_prog.

REQ_PROG:

REQ_PROG_1:

Moduł musi przechowywać program w postaci stałej pamięci ROM o rozmiarze $2^{\text{ADDR_WIDTH}}$ słów, gdzie każde słowo ma szerokość DATA_WIDTH bitów.

REQ_PROG_2:

Na wejściu a (adres) moduł ma przyjmować wartość w zakresie $[0, 2^{\text{ADDR_WIDTH}} - 1]$.

REQ_PROG_3:

Moduł ma zwracać na wyjściu out zawartość komórki pamięci o adresie a w trybie asynchronicznym.

9. Licznik rozkazów.

REQ_PC:

REQ_PC_1:

Moduł musi przechowywać bieżący adres rozkazu w rejestrze PC_count o szerokości W.

REQ_PC_2:

Wartość PC_count musi być aktualizowana w każdym cyklu zegara clk.

REQ_PC_3:

Jeśli skok_pc = 1, licznik ma wykonać skok do adresu podanego na wejściu adres_skok_pc.

REQ_PC_4:

W przypadku aktywacji sygnału skoku (skok_pc) oraz sygnału powrotu z podprogramu (skok_pc_stos), licznik musi załadować nową wartość adresu bezpośrednio ze stosu do pc oraz zinkrementować swoją nową wartość o 1.

REQ_PC_5:

W przypadku aktywacji sygnału skoku (skok_pc) oraz sygnału powrotu z przerwania (skok_pc_stos, reti_int_en), licznik musi załadować nową wartość adresu bezpośrednio ze stosu do pc.

REQ_PC_6:

Po odebraniu sygnału reset (rst) lub resetu licznika (ID_rst), zawartość akumulatora musi zostać wyzerowana w ciągu jednego cyklu zegarowego.

10. Port.

REQ_Port:

REQ_Port_1:

Moduł ma posiadać trzy porty logiczne (A, B, C), każdy o szerokości Port_rozm_data bitów.

REQ_Port_2:

Dla każdego portu istnieje rejestr kierunku DDRx.

Wartość bitu DDRx[i] = 1 oznacza, że pin i jest w trybie wyjścia (output).

Wartość bitu DDRx[i] = 0 oznacza, że pin i jest w trybie wejścia (input).

REQ_Port_3:

Rejestry DDR są zapisywane podczas zbocza narastającego zegara clk tylko jeśli wr_DDRx = 1. Adres rejestru DDR wybierany jest przez nr_P_DDRx.

REQ_Port_4:

Moduł posiada rejestry danych PORTx dla każdego portu. Rejestry PORTx są zapisywane podczas zbocza narastającego zegara clk tylko jeśli wr_PORTx = 1. Adres rejestru PORT wybierany jest przez nr_P_PORTx.

REQ_Port_5:

Wartości rejestrów DDR i PORT są resetowane przy rst = 1, na narastającym zboczku zegara clk:

REQ_Port_6:

Dla każdego pinu portu:

jeśli DDRx[i] = 1, to pin fizyczny in_out_X[i] jest sterowany przez wartość PORTx[i] (tryb output).

jeśli DDRx[i] = 0, to pin fizyczny in_out_X[i] jest w stanie wysokiej impedancji ('z'), a wartość logiczna pinu jest odczytywana na pin_mux.

REQ_Port_7:

Odczyt wartości pinów w trybie wejścia (DDR=0) jest realizowany asynchronicznie i jest dostępny na out w zależności od wybranego portu/pinu.

11. Przerwanie.

REQ_Przerwanie:

REQ_Przerwanie_1:

Moduł musi posiadać globalny bit zezwolenia na przerwanie przerwanie_en, który jest ustawiany sygnałem int_enable i zerowany sygnałem int_disable, przy narastającym zboczku zegara clk.

REQ_Przerwanie_2:

Moduł musi wykrywać narastające zbocze sygnału ext_int (przerwanie zewnętrzne) i zapamiętywać je w fladze int_a niezależnie od stanu przerwanie_en.

REQ_Przerwanie_3:

Moduł musi wykrywać narastające zbocze sygnału timer_int (przerwanie timera) i zapamiętywać je w fladze int_b tylko wtedy, gdy przerwanie_en = 1.

REQ_Przerwanie_4:

Priorytet przerwania jest następujący:

jeśli int_a = 1 oraz przerwanie_en = 1, to generowane jest przerwanie z wektorem 8'h02 (przerwanie zewnętrzne).

jeśli int_a = 0, a int_b = 1 oraz przerwanie_en = 1, to generowane jest przerwanie z wektorem 8'h04 (przerwanie timera).

REQ_Przerwanie_5

Jeśli przerwanie zewnętrzne (int_a) zostanie wykryte w czasie, gdy trwa przerwanie timera (int_b), to przerwanie zewnętrzne ma zostać zapamiętane i wykonane jako pierwsze po zakończeniu przerwania timera (priorytet).

REQ_Przerwanie_6

Po wygenerowaniu przerwania odpowiednia flaga (int_a lub int_b) musi zostać wyzerowana.

REQ_Przerwanie_7:

Reset (rst) musi zerować wszystkie wewnętrzne flagi i rejestry.

12. Stos.

REQ_Stos:

REQ_Stos_1:

Moduł musi posiadać pamięć stosu stos_pamiec o rozmiarze STOS_Rozm, gdzie każde słowo ma szerokość STOS_data_rozm.

REQ_Stos_2:

Moduł musi posiadać wskaźnik stosu stos_ptr, który wskazuje na następną wolną pozycję w stosie.

REQ_Stos_3:

Po resecie (rst = 1) stos musi zostać wyzerowany (stos_pamiec, stos_ptr, full=0, empty=1).

REQ_Stos_4:

W przypadku aktywacji sygnału push przy nieaktywnym sygnale pop, moduł musi zapisać dane wejściowe data_in na wierzchu stosu oraz zwiększyć wskaźnik stosu o jeden.

REQ_Stos_5:

Po wykonaniu operacji push, stos nie może być oznaczony jako pusty, a w przypadku zapisu ostatniego dostępnego elementu musi zostać ustawiona flaga full.

REQ_Stos_6:

W przypadku aktywacji sygnału pop przy nieaktywnym sygnale push, moduł musi zdjąć element z wierzchu stosu poprzez zmniejszenie wskaźnika stosu o jeden.

REQ_Stos_7:

Po wykonaniu operacji pop, stos nie może być oznaczony jako pełny, a w przypadku usunięcia ostatniego elementu musi zostać ustawiona flaga empty.

REQ_Stos_8:

Podczas operacji pop, jeżeli stos nie jest pusty, moduł musi wystawić na wyjściu data_out wartość elementu znajdującego się na wierzchu stosu.

REQ_Stos_9:

Jeżeli push = 1 i pop = 1 jednocześnie, to operacja jest nieokreślona (brak działania), moduł nie gwarantuje poprawnego zachowania.